

MySQL JOINS — Quick Reference

Combining data from multiple tables • General reference for any MySQL database

What is a JOIN?

Your data is split across separate tables. A **JOIN** lets you pull columns from multiple tables into one result set — like snapping puzzle pieces together.

Tables must share a **key column**: one table's **foreign key** matches the other's **primary key**. That shared column is the bridge.

Example: Products has a **CategoryID** column. Categories also has a **CategoryID**. JOIN on that to get the category name next to each product.

The four JOIN types

INNER Matching rows only — both sides must have a key

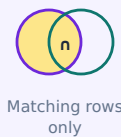
LEFT All rows from left table — NULLs where right has no match

RIGHT All rows from right table — NULLs where left has no match

FULL All rows from both tables (MySQL: simulate with UNION)

INNER JOIN — the most common

What it does: Returns only rows where a matching key exists in *both* tables. Any row without a match on either side is dropped.



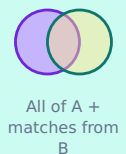
Use when: you want products that actually have a category, or orders tied to a real customer.

```
SELECT p.ProductID, p.ProductName,
       c.CategoryName
FROM   Products p
JOIN   Categories c
ON     p.CategoryID = c.CategoryID
ORDER BY c.CategoryName;
```

✓ Products with no category are excluded. Categories with no products are excluded.

LEFT JOIN — keep all of left table

What it does: Returns *all* rows from the first (left) table. If the right table has no matching row, right-side columns show **NULL**.



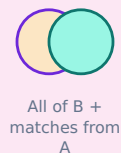
Use when: you want every product listed, even ones with no supplier on file yet.

```
SELECT p.ProductName,
       s.CompanyName
FROM   Products p
LEFT JOIN Suppliers s
ON     p.SupplierID = s.SupplierID
ORDER BY p.ProductName;
```

⚠ Products with no supplier still appear — CompanyName will be NULL.

RIGHT JOIN — keep all of right table

What it does: Returns *all* rows from the second (right) table. Left-side columns show **NULL** where there's no match.



Use when: less common — usually you can swap table order and use LEFT JOIN to get the same result.

```
SELECT p.ProductName,
       s.CompanyName
FROM   Products p
RIGHT JOIN Suppliers s
ON     p.SupplierID = s.SupplierID
ORDER BY s.CompanyName;
```

⚠ Suppliers with no products still appear — ProductName will be NULL.

FULL OUTER JOIN — keep everything

What it does: Returns *all* rows from both tables. NULLs fill in wherever there's no match on either side.



Use when: auditing — "show every record from both tables, matched or not."

```
-- MySQL: simulate with UNION
SELECT * FROM A LEFT JOIN B ON A.id = B.id
UNION
SELECT * FROM A RIGHT JOIN B ON A.id = B.id;
```

i MySQL has no native FULL OUTER JOIN. UNION removes duplicates; use UNION ALL to keep them.

MySQL JOINS — Page 2

Dot notation • Aliases • Line-by-line annotated example • Chaining multiple JOINS

What does `p.ProductName` actually mean?

When you JOIN two tables, MySQL loads columns from **both tables at the same time**. If both have a column with the same name — like `ID` — MySQL can't tell which one you mean without help. The **dot (.)** is that help. It points MySQL to the right table: `tableName.columnName` = "the column called X inside that specific table." When you use an alias like `p` for Products, `p.ProductName` means the same thing — just shorter.

Using full table names (verbose but valid):

```
SELECT Products.ProductName,  
       Categories.CategoryName  
FROM   Products  
JOIN   Categories  
  ON   Products.CategoryID  
      = Categories.CategoryID;
```

Same query using aliases (preferred):

```
SELECT p.ProductName,  
       c.CategoryName  
FROM   Products p  
JOIN   Categories c  
  ON   p.CategoryID  
      = c.CategoryID;
```

The key: `p` is a nickname you assign Products right after its name in FROM or JOIN. After that, `p.column` means "that column from Products." Same pattern: `c` = Categories, `s` = Suppliers, etc. It's shorter *and* unambiguous.

Line-by-line walkthrough — reading a full 3-table JOIN query

1	<code>SELECT p.ProductName,</code>	SELECT lists the columns you want in your results. <code>p.ProductName</code> = "ProductName from the table nicknamed p " — the alias you'll define on line 4.
2	<code>c.CategoryName,</code>	<code>c.CategoryName</code> is from a different table entirely (Categories, nicknamed c). The dot tells MySQL where to find it.
3	<code>s.CompanyName</code>	<code>s.CompanyName</code> is from yet another table (Suppliers, nicknamed s). Three columns, three tables, one query.
4	<code>FROM Products p</code>	FROM is your starting table. Writing <code>p</code> right after <code>Products</code> creates the alias — now <code>p</code> = Products everywhere in this query.
5	<code>JOIN Categories c</code>	First JOIN — attach the Categories table. Its alias is <code>c</code> .

Column Aliases (AS)

Rename a column in your output using **AS**. The result header shows your chosen name instead of the original column name.

```
SELECT  
  s.CompanyName  
  AS 'Shipper'  
FROM   Orders o  
JOIN   Shippers s  
  ON   o.ShipVia  
      = s.ShipperID  
ORDER BY 'Shipper';
```

⚠ Column alias works in **ORDER BY** — but **NOT** in **WHERE**.

Syntax Blueprint

```
SELECT a.col1, b.col2  
FROM   TableA a  
JOIN   TableB b  
  ON   a.sharedKey  
      = b.sharedKey  
WHERE  condition  
ORDER BY a.col1;
```

FROM	starting (left) table
JOIN / ON	snap + connect a table
WHERE	filter rows after joining
ORDER BY	sort the final result

Chaining Multiple JOINS

Need columns from 3+ tables? **Stack more JOIN ... ON pairs**. Each one snaps another table onto the growing result.

```
FROM   Products p  
JOIN   Categories c  
  ON   p.CategoryID  
      = c.CategoryID  
JOIN   Suppliers s  
  ON   p.SupplierID  
      = s.SupplierID
```

Pattern: FROM → JOIN/ON → JOIN/ON → as many as you need. Each JOIN gets its own alias and its own ON condition.